복사전달 특강
Special Topics in Radiative Transfer

Lecture 14
2026 June 2 (Tuesday), 1PM

updated on 06/01

Based on: Numba documentation, Seon 2009 (PKAS), Amanatides & Woo 1987, Revelles et al. 2000

선광일 (Kwangil Seon)
KASI / UST

Outline – Numba, Fast Ray Traversal, and AMR Grids

Why Numba?

Monte Carlo radiative transfer hot loops are deeply nested, scalar-heavy, and not vectorizable in NumPy — exactly the workload that pure-Python loops execute slowest on.

Numba's job. Compile a Python function to machine code via LLVM, ahead of the first call. A well-placed `@njit` delivers C-level performance on the same source code, while letting the orchestrating code stay in plain Python.

The catch. Inside a `@njit` function, every callee must itself be Numba-compilable: another `@njit` function, or a NumPy / built-in operation that Numba understands natively. A forgotten decorator on a helper is the single most common cause of failure.

The JIT boundary rule

Three lines summarize where to put `@njit`:

1. The function that contains the *hot loop* (where most of the CPU time is spent) must carry `@njit`.
2. Every helper called from inside that hot loop must also carry `@njit`.
3. The outer Python wrapper that orchestrates calls into the JIT'd code does *not* need `@njit`.

Boundary cost. Crossing the Python / JIT boundary is a few µs per call. If each JIT call does more than a few ms of work, the boundary cost is invisible. A tight Python loop that calls a fast JIT function 10^6 times *is* dominated by the boundary; the fix is to lift the loop into the JIT region (§ JIT outer wrapper).

Imports and decorator options

The minimum import is just the decorator:

```
from numba import njit
```

For parallel loops, also import `prange`:

```
from numba import njit, prange
```

The decorator options used in this repository:

- `@njit(cache=True)` — write the compiled binary to `__pycache__/*.nbc`; next session skips compilation. Recommended when the signature is stable.
- `@njit(parallel=True)` — enable Numba's parallel transforms; this is what makes `prange` multithread.
- `@njit(fastmath=True)` — relax IEEE-754 (allow reassociation, no signed zero, ...). Use sparingly: it can change numerical results.

A minimal example: π by dart-throwing

```python
import numpy as np
from numba import njit

@njit(cache=True)
def sample_point():                          # helper: must be @njit
    x = 2.0 * np.random.random() - 1.0
    y = 2.0 * np.random.random() - 1.0
    return x, y

@njit(cache=True)
def count_inside(N, seed=42):                # hot loop: must be @njit
    np.random.seed(seed)
    count = 0
    for _ in range(N):
        x, y = sample_point()
        if x*x + y*y < 1.0:
            count += 1
    return count

def estimate_pi(N, seed=42):                 # plain-Python wrapper
    return 4.0 * count_inside(N, seed) / N
```

`sample_point` is inlined into the hot loop; `estimate_pi` crosses the JIT boundary once.

Compilation cost and warm-up

The *first* call to a `@njit` function in a Python process triggers LLVM compilation: ~ 0.1 s for a trivial helper, several seconds for `parallel=True` code. Subsequent calls execute the compiled binary directly. With `cache=True`, the next session loads the cached binary in a few tens of ms.

Warm-up is cosmetic, not load-bearing

```
print("Warming up JIT ...")
_ = my_jitted_function(small_problem_size, seed=0)
```

Compilation happens once per session in any case. The warm-up call only shifts *when* it happens — out of the timed region and into a labelled prelude. Output is bit-for-bit identical with or without warm-up; total wall time is unchanged.

When to skip warm-up. Production scripts and CI jobs that don't time anything do not benefit. The real persistent speedup across sessions is `cache=True`, not warm-up; with caching, even the warm-up call itself is essentially free.

When to JIT the outer wrapper

Two situations only:

1. **The outer loop is itself the hot loop.** If a Python `for` calls a very fast JIT function many times, the per-call boundary cost ($\sim 1\,\mu s$) dominates. Putting `@njit` on the wrapper folds the loop into the JIT region and removes the crossings.
2. **You want Numba parallelism.** A `prange` loop only parallelises inside `@njit(parallel=True)`. The outer wrapper is the natural place to declare it.

The parallel pattern, extending the π example to many independent runs at once:

```
from numba import njit, prange

@njit(cache=True, parallel=True)
def count_inside_many(N_per, N_runs, base_seed=42):
    counts = np.empty(N_runs, dtype=np.int64)
    for i in prange(N_runs):                          # threaded
        counts[i] = count_inside(N_per, base_seed+i)  # inner @njit
    return counts
```

The outer JIT is needed because of `prange`; the inner `count_inside` remains `@njit` because it is called from JIT code.

Common pitfalls

1. **Every callee inside `@njit` must also be `@njit`.** The single most common compile-time error. Numba's error message points at the offending callee.
2. **No Python objects in hot paths.** Lists, dicts, and arbitrary classes are unsupported (or use Numba's slower typed containers). Stick to NumPy arrays and scalars.
3. **Avoid allocations in the inner loop.** Returning a tuple of scalars is allocation-free; returning a small NumPy array per call allocates on every iteration and shows up immediately in profiles. The idiom is to pass and return scalar components.
4. `np.random` **inside** `@njit` **uses Numba's own RNG state**, independent of NumPy's global generator. To reproduce a run, call `np.random.seed(seed)` from *inside* the JIT function.
5. `cache=True` **can serve a stale binary.** If a function still runs the old version after editing, delete `__pycache__/*.nbc` and rerun.

Why ray traversal matters in MCRT

For every free flight and every peeling-off direction, an MCRT code integrates the optical depth

$$\tau(\ell) = \int_0^{\ell} \kappa(x, y, z)\, \mathrm{d}\ell$$

along a straight photon path through a piecewise-constant medium. In an optically thick cloud this kernel runs many times per packet and is the *single largest contributor* to the per-photon cost.

Two algorithms, same answer.

- **Baseline** (Seon 2009): recompute $\delta X, \delta Y, \delta Z$ from the current position each iteration. Three subtractions and three divisions per step.
- **Fast Voxel Traversal** (Amanatides & Woo 1987): incremental Digital Differential Analyzer (DDA). Two comparisons + one addition + one integer step per iteration.

Both visit the same voxels in the same order. The DDA simply spends much less arithmetic per step.

Grid setup

Volume $[X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}] \times [Z_{\min}, Z_{\max}]$ partitioned into $N_x \times N_y \times N_z$ uniform cells:

$$\Delta X = \frac{X_{\max} - X_{\min}}{N_x}, \quad \Delta Y = \frac{Y_{\max} - Y_{\min}}{N_y}, \quad \Delta Z = \frac{Z_{\max} - Z_{\min}}{N_z}.$$

Cell (i,j,k) corner: $X_i = (i-1)\Delta X + X_{\min}$, opposite corner $(X_{i+1}, Y_{j+1}, Z_{k+1})$. Opacity per unit length

$$\kappa(i,j,k) \;\equiv\; \sigma\, n(\bar{x}, \bar{y}, \bar{z})$$

is constant on each cell. Photon at $(X_{\text{cur}}, Y_{\text{cur}}, Z_{\text{cur}})$ in cell (i,j,k), moving along $\mathbf{n} = (n_x, n_y, n_z)$, $|\mathbf{n}| = 1$. Find step length L with

$$\int_0^L \kappa(x(\ell), y(\ell), z(\ell))\, \mathrm{d}\ell \;=\; \tau_{\text{target}} \;=\; -\ln \xi\,.$$

Because κ is piecewise constant, $\tau(\ell) = \sum_p \kappa(i_p, j_p, k_p)\, \delta L_p$.

The cell-crossing geometry

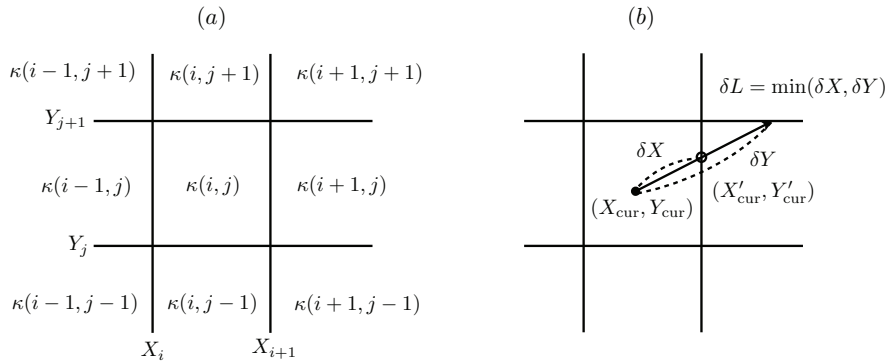


Figure 1: Reproduced from Seon (2009), Fig. 3. **(a)** 2D sketch of the Cartesian grid; cell (i,j) stores $\kappa(i,j)$ and is bounded by the planes $x = X_i, X_{i+1}$ and $y = Y_j, Y_{j+1}$. **(b)** One traversal step: from $(X_{\text{cur}}, Y_{\text{cur}})$ along direction (n_x, n_y), the parametric distances to the next x- and y-faces are δX and δY; the actual step is $\delta L = \min(\delta X, \delta Y)$, so the ray exits through whichever face it reaches first.

Baseline algorithm (Seon 2009)

Distances to the next axis-aligned faces from $(X_{\text{cur}}, Y_{\text{cur}}, Z_{\text{cur}})$:

$$\delta X = \begin{cases} (X_{i+1} - X_{\text{cur}})/n_x, & n_x > 0, \\ (X_i - X_{\text{cur}})/n_x, & n_x < 0, \end{cases} \qquad \delta Y, \delta Z \text{ analogously,}$$

$$\delta L = \min(\delta X, \delta Y, \delta Z).$$

Advance the photon:

$$X_{\text{cur}} \mathrel{+}= n_x \delta L, \quad Y_{\text{cur}} \mathrel{+}= n_y \delta L, \quad Z_{\text{cur}} \mathrel{+}= n_z \delta L.$$

Increment the cell index along the winning axis ($i \mathrel{+}= \pm 1$, etc.). Accumulate $\tau_{\text{acc}} \mathrel{+}= \kappa(i,j,k)\,\delta L$ and repeat. When $\tau_{\text{acc}} + \kappa\,\delta L \geq \tau_{\text{target}}$, the interaction lies inside the current cell; solve the partial step

$$\delta L^* = \frac{\tau_{\text{target}} - \tau_{\text{acc}}}{\kappa(i,j,k)}, \qquad x' = X_{\text{cur}} + n_x \delta L^*, \ \ldots$$

Cost per step: $3\,\text{div} + 3\,\text{sub} + 4\,\text{mul} + 4\,\text{add} + 2\,\text{cmp}$, dominated by the three divisions.

The DDA key observation

Along a fixed direction $\mathbf{n}$, the parametric distance between consecutive x-faces is a *constant*:

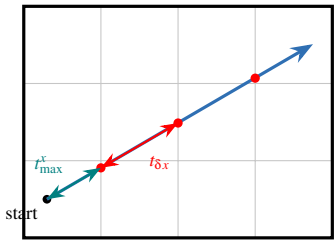
$$\boxed{t_{\delta x} \equiv \frac{\Delta X}{|n_x|}} \qquad \text{(and similarly } t_{\delta y}, t_{\delta z}\text{)}.$$

So once the parametric distance to the *first* x-face is known, every subsequent x-face distance follows by repeated addition of $t_{\delta x}$. No subtraction or division is ever required inside the loop.

Analogy. The baseline asks a passer-by for directions at *every* intersection (recompute from scratch); the DDA plans the whole route once, then merely counts city blocks.

Ray parameterization. Write $\mathbf{p}(t) = (X_{\text{cur}}, Y_{\text{cur}}, Z_{\text{cur}}) + t\,\mathbf{n},\ t \geq 0$. Because $|\mathbf{n}| = 1$, t is the physical arc length along the ray; δL_p is just the difference of two consecutive t values.

DDA in one picture: $t_{\max}$ and t_{δ}



Pick a direction. The distance to the *first* x-face is $t^x_{\max}$ (teal). Every *next* x-face is a *constant* step

$$t_{\delta x} = \frac{\Delta X}{|n_x|}$$

farther along the ray (red dots, equal t-gaps). So updating to the next x-crossing is just

$$t^x_{\max} \mathrel{+}= t_{\delta x},$$

one addition — no division. The same holds for y and z; each step takes whichever face comes first.

DDA: one-time initialization

At the start of a ray, compute and store:

$$\text{step}_x = \text{sign}(n_x), \quad \text{step}_y = \text{sign}(n_y), \quad \text{step}_z = \text{sign}(n_z),$$

$$t^x_{\max} = \begin{cases} (X_{i+1} - X_{\text{cur}})/n_x, & n_x > 0 \\ (X_i - X_{\text{cur}})/n_x, & n_x < 0 \end{cases} \quad (\text{and } t^y_{\max}, t^z_{\max}),$$

$$t_{\delta x} = \Delta X/|n_x|, \quad t_{\delta y} = \Delta Y/|n_y|, \quad t_{\delta z} = \Delta Z/|n_z|.$$

Cost: $6\,\text{div} + 3\,\text{sub}$ *once per ray*. This is the same arithmetic the baseline does on *every* step — the DDA amortizes it across the whole ray.

Axis components with $n_x = 0$ set the corresponding $t_{\max}$ and t_δ to $+\infty$ so they are never selected as the minimum.

DDA: inner loop

Maintain t_{cur} (initially 0) and τ_{acc} (initially 0). The 3D loop body is a three-way minimum implemented as cascaded comparisons:

```
if tMaxX < tMaxY:
    if tMaxX < tMaxZ:        axis, tHit = 'X', tMaxX
    else:                    axis, tHit = 'Z', tMaxZ
else:
    if tMaxY < tMaxZ:        axis, tHit = 'Y', tMaxY
    else:                    axis, tHit = 'Z', tMaxZ

dL       = tHit - t_cur
dtau     = kappa[i, j, k] * dL
if tau_acc + dtau >= tau_target:           # interaction inside cell
    dL_star = (tau_target - tau_acc) / kappa[i, j, k]
    return reconstruct_position(t_cur + dL_star)

t_cur    = tHit                            # advance running t
tau_acc += dtau
if   axis == 'X': i += stepX;  tMaxX += tDeltaX     # one add, no div
elif axis == 'Y': j += stepY;  tMaxY += tDeltaY
else:             k += stepZ;  tMaxZ += tDeltaZ
```

Per-step cost: 2 cmps, 1 sub (δL), 1 mul + 1 add (τ), 1 add ($t_{\max}$ refresh), 1 integer step. *No divisions inside the loop.*

How much faster? – operation counts

Operation per voxel-traversal step	Baseline	DDA
Divisions	3	0
Subtractions	3	1
Additions	3	2
Multiplies	4	1
FP comparisons	2	2
Integer increments	1	1
Approx. FLOP / step	$\sim$13 (incl. 3 div)	$\sim$6 (no div)

Cycle model. With pipelined ops at ~ 1 cycle and division latency $c_{\text{div}} \approx 25$ cycles (double precision `vdivpd`),

$$C_{\text{base}} \approx 3\times 25 + 10\times 1 = 85\,\text{cycles}, \quad C_{\text{DDA}} \approx 0 + 6\times 1 = 6\,\text{cycles}.$$

Per-step speedup $\boxed{C_{\text{base}}/C_{\text{DDA}} \sim 10\text{–}15}$. Where division is fully pipelined the gap narrows to $\sim$ 3–5; it never depends on ray length or optical depth.

Break-even and asymptotic cost

Initialization cost. The DDA pays one-time $6\,\mathrm{div} + 3\,\mathrm{sub} \approx 150$ cycles, vs. essentially zero for the baseline.

Per-step saving. $C_{\mathrm{base}} - C_{\mathrm{DDA}} \approx 80$ cycles.

Break-even.

$$M_{\mathrm{BE}} \approx \frac{150}{80} \approx 2 \text{ steps.}$$

Any ray that crosses three or more voxels favors the DDA. In MCRT the typical photon crosses many cells before its first scattering, so initialization is irrelevant in practice.

Asymptotic. Both algorithms visit the same $M = O(\tau/(\kappa\,\Delta X))$ voxels per ray, so total cost is $M\,C + C_{\mathrm{init}}$. The DDA shrinks C by an order of magnitude and leaves M unchanged. In a code whose hot loop is the voxel traversal — most pure-scattering codes and the optically thick limit of absorption-dominated codes — the overall MCRT wall-clock improves by a similar factor.

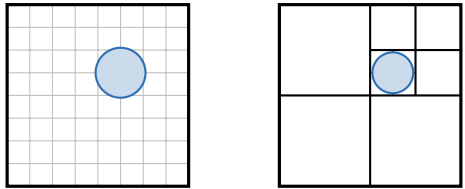
Practical notes

- **Snap-to-face is automatic in the DDA.** The cell index changes by an integer ± 1; the physical position is only reconstructed at the end from $t_{\mathrm{cur}} + \delta L^*$. There is no opportunity for the index and position to disagree — a free robustness gain over the baseline's manual snapping.
- **Forced first scattering.** (Witt 1977) Run either algorithm with an unbounded target until the ray exits the volume and read off the accumulated τ_1. The DDA is the natural choice because every ray-to-boundary crosses many voxels.
- **Peeling-off optical depth.** (Yusef-Zadeh et al. 1984) Once per scattering, per observer direction. This is the most traversal-heavy phase of MCRT and the place where the DDA speedup pays back many times over.
- **Beyond Cartesian.** The same template — parametric distances to the next boundaries, take the minimum, advance one axis — generalizes to nonuniform axis-aligned grids and to spherical or cylindrical grids (one quadratic solve per shell crossing).

Why adapt the mesh? (the AMR motivation)

A uniform grid has a single knob: one cell size, the same everywhere. To resolve a small dense clump you must refine the *entire* box — paying N^3 cells, almost all of which sit in empty voids contributing nothing.

Adaptive Mesh Refinement (AMR) breaks that tie: spend small cells only where the physics is sharp (dense gas, steep gradients) and leave big cells in the smooth voids.



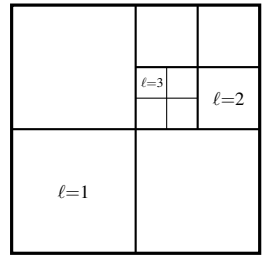
uniform: 64 cells AMR: 10 cells

Figure 2: Same core resolution, two ways. Left: a uniform grid spends 64 cells, nearly all in empty space. Right: a graded (quad/oc)tree reaches the *same* fine resolution at the core with only 10 cells.

The octree: recursive 8-way subdivision

Begin with the whole box as the *root* cell. Wherever more resolution is needed, split a cell into **8 equal children** (octants) and recurse. (In 2D the analog is a *quadtree*, 4 children — the figures use 2D for clarity.)

- A **leaf** is an undivided cell — it stores the physical data (κ, T, $\mathbf{v}$).
- An **internal** cell has been split and only points to its children.
- Level $\ell \Rightarrow$ size $\Delta_\ell = L_{\text{box}}/2^\ell$. *All sizes are powers of two* — keep this in mind for the DDA discussion.
- Child index $= 1 + i_x + 2i_y + 4i_z$, where i_x=1 if $x \geq$ cell center, else 0 (same for i_y, i_z).



Finding a cell — and its neighbor

Point location ("which leaf holds (x, y, z)?"). Descend from the root: at each node pick the octant by comparing to the cell center; stop at a leaf. Cost $O(\text{depth}) = O(\log N)$ — not the $O(1)$ index arithmetic a uniform grid enjoys.

Neighbor finding is the crux of traversal. The cell across a face can be (1) the *same* size, (2) *bigger* (a coarse neighbor), or (3) *split* into finer faces — then you must descend to pick the right child.

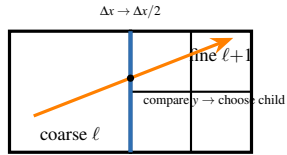


Figure 3: A ray leaving a coarse cell (level ℓ) enters a region refined to level $\ell+1$. At the shared face the code halves the cell size and descends into the correct finer child by comparing the transverse coordinate.

LaRT precomputes a **face-neighbor table** (6 entries/cell), so the jump is $O(1)$ plus a short descent only when the neighbor is finer.

AMR ray traversal: a per-cell baseline solve

Because the cell size *changes* from cell to cell, the DDA's constant increment t_δ no longer holds. Each cell therefore falls back to the **baseline** solve (§ Baseline algorithm): from the current position, distance to each of the 6 faces, then the minimum.

```
# one traversal step in an AMR leaf (cf. amr_cell_exit)
t[1] = (cx + h - x)/kx    ;  t[2] = (cx - h - x)/kx      # +x, -x faces
t[3] = (cy + h - y)/ky    ;  t[4] = (cy - h - y)/ky      # +y, -y
t[5] = (cz + h - z)/kz    ;  t[6] = (cz - h - z)/kz      # +z, -z
iface  = argmin(t)        ;  t_exit = t[iface]           # 6-way min
x += t_exit*kx ; y += t_exit*ky ; z += t_exit*kz         # advance
il   = amr_next_leaf(icell, iface, x, y, z)              # neighbor table
```

Per-step cost: 3 sub + 3 div + 6-way min (the face solve), *plus* the neighbor-table lookup and any descent. Those 3 divisions are exactly the baseline cost the uniform DDA had hoisted out of the loop.

Can we run a DDA on an octree?

Yes — but it is more work, and three frictions appear at every *level change*:

1. t_δ **changes** — but only by a factor of two. Rescale $t_\delta \to t_\delta/2$ (refine) or $2\,t_\delta$ (coarsen): a shift, *not* a division. The easy part.
2. **Neighbor finding across levels** — the real cost. A finer neighbor means descending to choose the right child; a coarser one means popping up the tree.
3. **Child selection** — compare the transverse coordinate to the child mid-plane. Comparisons, not divisions.

Known recipes.

- *Parametric octree traversal* (Revelles, Ureña & Lastra 2000): walk the tree using only the ray's entry/exit t-values; divisions happen *once* at the top, the rest is adds and compares — a DDA in spirit.
- *Hybrid:* run the plain DDA while the ray stays at one level; rescale t_δ ($\times 2$) and descend only at refinement boundaries. Most of the gain, a small code change.

Why the DDA payoff shrinks on an octree

On a *uniform* grid the next cell is just `index += step` — *free*. So removing the 3 divisions is the *whole* game, and the DDA wins by the full $\sim$3–15$\times$ (the operation-count table earlier).

On an *octree* the per-step cost splits in two:

$$C_{\text{step}} \approx \underbrace{C_{\text{face-solve}}}_{\text{DDA removes the 3 div}} + \underbrace{C_{\text{neighbor}} + C_{\text{descent}}}_{\text{DDA cannot remove}}.$$

The second term — table lookup, tree descent at level changes, cache misses on scattered leaf arrays — is often the *larger* one. So even an ideal octree-DDA gains less than the uniform DDA, while costing real code complexity and floating-point robustness at T-junctions.

That trade-off is why LaRT uses the incremental DDA for its Cartesian grid, but a baseline face-solve + neighbor table for AMR.

Uniform/DDA vs. AMR — when each wins

	Uniform grid (DDA)	**AMR (octree)**
Cell identity	integer (i,j,k)	tree / leaf index
Next cell	`index += step` (free)	neighbor table + descent
Distance to face	1 add (incremental)	3 sub + 3 div (re-solved)
Memory layout	dense, contiguous	flat arrays, indirection
Memory size	every cell, even voids	only where refined
Per-step cost	lowest	higher
Best for	homogeneous media	multi-scale, clumpy media

Bottom line. AMR does not make each *step* faster — it makes the *number* of cells and steps M far smaller when the medium spans many scales. Force an AMR grid to be uniform and you keep all of AMR's per-step overhead while throwing away its only advantage; a uniform DDA then beats it every time.

Take-home messages

Part I — Numba. Put `@njit` on the hot loop and every helper it calls; leave the orchestrating wrapper in plain Python unless it is itself the hot loop or wraps a `prange`. Use `cache=True` for persistent speedup across sessions; warm-up is only cosmetic.

Part II — Ray traversal. The baseline algorithm (Seon 2009) is correct but wastes three divisions per voxel crossing. The Amanatides–Woo DDA hoists the divisions into a one-time initialization, leaving the inner loop with only comparisons and an addition. The per-step speedup is $\sim$10–15$\times$ in the division-bound regime, $\sim$3–5$\times$ when division is well-pipelined — a fixed multiplier on every voxel crossing.

Part III — AMR. An octree adapts the cell size to the medium, slashing the cell count in multi-scale problems. Its varying cell size rules out the constant-increment DDA, so traversal reverts to a per-cell baseline solve plus a neighbor-table jump — worthwhile only because there are far fewer cells to cross.

Together. A Numba-compiled DDA traversal is the natural inner kernel for any production-grade MCRT code. The Python / JIT boundary is crossed once per packet (or once per ray for peeling-off), and the per-voxel arithmetic is reduced to a handful of cycles.

References

- Numba documentation, `https://numba.readthedocs.io/`.
- Seon, K.-I. 2009, "Monte-Carlo simulation of the dust scattering," *PKAS* **24**, 43–51.
- Amanatides, J. and Woo, A. 1987, "A fast voxel traversal algorithm for ray tracing," *Proc. Eurographics '87*, 87, 3–10.
- Revelles, J., Ureña, C., and Lastra, M. 2000, "An efficient parametric algorithm for octree traversal," *J. of WSCG* **8**, 212–219.
- Teyssier, R. 2002, "Cosmological hydrodynamics with adaptive mesh refinement (RAMSES)," *A&A* **385**, 337–364.
- Witt, A. N. 1977, "Multiple scattering in reflection nebulae. I. A Monte Carlo approach," *ApJS* **35**, 1.
- Yusef-Zadeh, F., Morris, M., and White, R. L. 1984, "Bipolar reflection nebulae: Monte Carlo simulations," *ApJ* **278**, 186.